17. List Interface in Java — Quick Notes

Definition

- Child interface of `Collection`
- Ordered collection
- Supports index-based access
- Allows duplicate elements

List Hierarchy

Iterable
↓
Collection
↓
List

Main Implementations

Class	Internal DS	Notes
ArrayList	Dynamic Array	Most used
LinkedList	Doubly Linked List	Good for insert/delete
Vector	Dynamic Array	Thread-safe, Legacy
Stack	Extends Vector	Legacy

Important Point

Recommended
✅ ArrayList
❌ Vector (Legacy)
❌ Stack (Use ArrayDeque)

Why List Interface?

Provides common methods for: ArrayList, LinkedList, Vector
So code can work with any implementation.

Example:

List<Integer> list = new ArrayList<>();
Later can change to: List<Integer> list = new LinkedList<>();
No other code changes needed.

List Specific Methods

These methods are NOT present in Collection interface.

---

get()

Purpose: Get element at index.

List<Integer> list = Arrays.asList(10,20,30);
System.out.println(list.get(1));

Output: 20

---

set()

Purpose: Replace element at index.

list.set(1, 50);
Before: [10,20,30]
After: [10,50,30]

---

indexOf()

Purpose: Returns first occurrence index.

List<Integer> list = Arrays.asList(10,20,30,20);
System.out.println(list.indexOf(20));

Output: 1

Important
Returns:First occurrence only

---

remove(index)

Purpose: Remove item using index.

list.remove(1);
Before: [10,20,30]
After:[10,30]

Difference Between remove()

Method	Removes
remove(Object)	Element
remove(index)	Index position

Example:

list.remove(20);
Remove element 20.

list.remove(1);
Remove element at index 1.

ListIterator

Enhanced version of Iterator.

Iterator
Supports:

- hasNext()
- next()
- remove()

Direction: Forward Only →

ListIterator
Supports:

- hasNext()
- next()
- hasPrevious()
- previous()
- remove()

Direction:
← Backward & Forward →

ListIterator Example
```
ListIterator<Integer> it = list.listIterator();

Forward:
while(it.hasNext())
  System.out.println(it.next());

Backward:
while(it.hasPrevious())
  System.out.println(it.previous());
```

---

listIterator(index)
Purpose: Start traversal from given position.

```
ListIterator<Integer> it = list.listIterator(2);
Starts from: index = 2
```

Abstract Classes (Internal)
Used internally by Java Collections.

Examples:

- AbstractList
- AbstractSequentialList

Purpose:

- Provide common implementation
- Easier to create new List classes

Not important for interviews.

Advantages of List Interface

1. Common API
Same methods for: ArrayList, LinkedList, Vector

2. Easy Implementation Change
List<Integer> list = new ArrayList<>();
Can become: List<Integer> list = new LinkedList<>();

without changing business logic.

3. Code Flexibility
Program against interface, not implementation.

Important Quick Points

Topic	Point
List	Ordered collection
Duplicates	Allowed
get(i)	Fetch by index
set(i,e)	Replace element
indexOf()	First occurrence
remove(index)	Remove by position
ListIterator	Forward + Backward
ArrayList	Most commonly used
Vector/Stack	Legacy classes
Best Practice	Use `List` reference type